

THE QUANT CLUB · IIT (BHU) VARANASI

HIDDEN HAND

Problem Statement

Different hands. Same rules. Only the sharpest minds win.

Dates	5 to 8 October 2026
Format	Teams of 3, each with a mentor
Arena	Link on the WhatsApp community
Kit	Download from the arena (<i>Kit</i> , top bar)
Questions	Your mentor, then the WhatsApp community
Instagram	@thequantclub.itbhu

Analyze. Develop. Outplay.

Contents

1	The game	2
2	A round	2
2.1	Moves	2
2.2	Trading	3
3	A real deal	3
4	Your bot	3
5	House bots	4
6	The arena	4
7	What you submit	5
8	Evaluation	5
9	Rules	5
10	Quick start	6
11	FAQ	6
11.1	Glossary	6
12	Timeline	7

Hidden Hand

You will write a Python bot that plays a card-trading game against other teams' bots. Every bot holds a hidden hand of cards, and they trade a stock whose value is set by everyone's cards. No finance background is needed. If you can write a Python class and you know what an average is, you can start today.

Build	One Python file: a <code>Bot</code> class with three methods
Play	Two bots, zero-sum, 5 rounds per deal, hundreds of deals per fight
Win	Highest score: your PnL (profit and loss) deal by deal, on a square-root scale so luck cannot decide it
Allowed	Python 3.10+, standard library only, AI tools
Submit	Upload to the arena (link on the WhatsApp community). Results appear within minutes

1 The game

The house shuffles one ordinary 52-card deck. Each card is worth its rank: A = 1, 2 to 10 at face value, J = 11, Q = 12, K = 13.

- **You** get 5 cards. Only you see them.
- **Your opponent** gets 5 cards. Only they see them.
- **The board** is 5 more cards, face down. One is turned face up at the start of each of rounds 2, 3, 4 and 5. The last one stays hidden until the end.

The stock you trade is worth **the sum of all 15 cards**: your hand, their hand and the whole board, which averages 105. At the end every card is turned over and every trade is paid out. If you bought below the final sum or sold above it, you made money, and your opponent lost exactly that amount.

2 A round

Every deal has five rounds. In each round a board card is turned (rounds 2 to 5), both bots choose a move at the same time, one bot quotes a price and the other trades on it, and then the roles swap.

2.1 Moves

Move	Cost	What it does
nothing	0	–
"PEEK"	3	You privately see one random card from the opponent's hand. They learn that you peeked, but not which card.
("SWAP", i)	0	Throw away card i of your hand face up and draw a new card that only you see. Once per deal.

A swap changes the stock's value. If you hold 4 lots and swap a 2 for a Q, everything you hold is worth 10 more per lot. Your opponent sees the 2 go but not what replaced it.

2.2 Trading

The *maker* names a single price. The *taker* may buy up to 2 lots at price + 2, sell up to 2 lots at price − 2, or pass. Every round each bot quotes once and trades once. Nobody may hold more than 10 lots long or short.

The maker's price is a clue. A maker usually quotes what it thinks the stock is worth, and that depends on its hand. The taker's choice to buy or sell is a clue too. Reading these clues is how you learn the hidden hand.

Fairness. Every deal is played twice with the hands swapped. Two copies of the same bot always score exactly zero.

3 A real deal

House bot `sharp` against house bot `reader`. Reproduce it with `python backtester.py sharp reader --deals 1 --show`.

sharp holds	$Q\heartsuit$ $4\clubsuit$ $5\diamondsuit$ $10\spadesuit$ $6\heartsuit$	(sum 37)
reader holds	$7\spadesuit$ $4\heartsuit$ $3\clubsuit$ $7\clubsuit$ $9\heartsuit$	(sum 30)
Board, face down	$A\clubsuit$ $10\clubsuit$ $J\heartsuit$ $J\clubsuit$ $8\clubsuit$	

Round 1. `sharp` reckons $37 + 10 \times 6.96 = 106.6$ (ten unseen cards at the average unseen rank) and quotes that. `reader` thinks the stock is cheaper and sells 2 at 104.57. `reader` then quotes 101.04, and `sharp` buys 2 at 103.04.

Round 2. $A\clubsuit$ turns. `sharp` is long, so it swaps its $4\clubsuit$ and draws a $7\diamondsuit$: +3 on everything it holds. It keeps buying from `reader`.

Rounds 3–5. Prices close in and trading stops. `sharp` finishes long 10.

Settlement. $40 + 30 + 41 = 111$. `sharp` bought at about 103 and makes +82. The same deal is then played again with the hands swapped.

4 Your bot

```
class Bot:
    def move(self, obs):          # once per round, before trading
        return None              # or "PEEK", or ("SWAP", index)

    def quote(self, obs):         # you are the maker
        return 104.5             # they buy at 106.5 or sell at 102.5

    def respond(self, obs, price): # they quoted `price`
        return 2                 # +n buy n, -n sell n, 0 pass
```

A fresh `Bot()` is built for every deal. `obs` holds everything you are allowed to know:

Field	Meaning
round	1 to 5
hand	your 5 cards now, as ranks 1 to 13
board	board cards turned so far
opp_known	opponent cards you peeked that they still hold
unseen	<code>unseen[r-1]</code> = copies of rank <code>r</code> you have not seen anywhere (card counting, done for you)
position, cash, spent	lots held (+ long, – short), money from trades, paid for moves
tape	every quote so far: <code>(round, "me"/"opp", price, qty)</code>
discards	every swapped-out card: <code>(round, "me"/"opp", rank)</code>
peeks	every peek: <code>(round, "me"/"opp")</code>

A crash, an unusable return value or a call over 50 ms counts as doing nothing for that action. If your `quote()` fails, the house quotes 105 for you.

5 House bots

Three reference bots ship with the kit and play in the arena. You may copy anything from them.

Bot	What it does
naive	Counts cards: my cards + unseen cards $\times$ average unseen rank. Ignores the opponent and never moves. The starter bot is naive.
reader	<code>naive</code> , plus it estimates the opponent's hand from their last price
sharp	<code>reader</code> , plus it swaps a low card when long (a high one when short) and sizes trades by edge. Never peeks

They are far from the best possible bot. Things they never try: peeking, learning from the opponent's trades, expecting the opponent to swap, and quoting a price that hides their own hand.

Test properly. One deal can swing by 60 or more. Run 1,000 deals (about two seconds) on two or three seeds before you believe a change helped.

6 The arena

You submit and see results on the arena; its link is posted on the WhatsApp community.

1. Log in with the team key the organisers give you.
2. Upload your `.py` file. It is checked at once; a failed check shows why and does not count against your upload limit.
3. Your newest working file then plays every other team's newest file and the house bots on hidden deals. The live standings rank the score (see Evaluation), and you can watch the first deal of any fight, round by round.

Code is never shown to anyone; results and replays are public. You may upload once every 10 minutes, and your newest working file is your entry.

The final. After the deadline every entry plays every other on a different hidden set of deals. Only the final decides the result.

7 What you submit

One `.py` file per team, made from `starter_bot.py`, which:

- starts with these three lines, filled in:

```
# Team Name: Your Team Name
# Members: Name 1 (Roll No.), Name 2 (Roll No.), Name 3 (Roll No.)
# Mentor: Your Mentor's Name
```

- defines Bot with `move`, `quote` and `respond`;
- imports only `math`, `random`, `statistics`, `collections`, `heapq`, `bisect`, `itertools`, `functools`, `typing`;
- passes `python backtester.py --validate your_bot.py`;
- keeps nothing from one deal to the next.

8 Evaluation

You are ranked on a **score** built deal by deal, so that one lucky deal cannot decide anything.

1. Every deal is played twice with the hands swapped. Add your PnL from the two plays and call it p . The luck of the cards mostly cancels here.
2. Take the signed square root of p : a deal pair won by 100 counts +10, one won by 25 counts +5, one lost by 16 counts −4.
3. Your score is the average of these over every deal pair of every final fight:

$$\text{score} = \frac{1}{N} \sum_{i=1}^N \text{sign}(p_i) \sqrt{|p_i|}$$

A single huge win, say a lucky last card while you are long 10, counts for far less than its raw PnL, while winning deal after deal counts in full. Tested over 12 sets of 500 deals, this separates a better bot from a worse one far more reliably than raw PnL. `backtester.py` prints your score, so you can tune for exactly what is ranked.

Ties are broken on PnL per deal. Every bot plays every other bot and the three house bots the same number of deals. House bots do not take a place.

9 Rules

1. Your bot is your team's own work. Discussing ideas is fine; sharing code or another team's tuned numbers is not. The starter and house bots are free to copy.
2. AI tools are allowed. Any member must be able to explain any part of the file.
3. Mentors guide; they do not build. A bot written or tuned by a mentor is disqualified.
4. Play the game, not the system. Reading files, using the network or keeping state between deals gets the file rejected. Report loopholes; do not use them.

5. One team, one entry. A member may be in only one team.
6. The organisers' decision is final. Ask on the group before building on a guess.

10 Quick start

Download the kit from the arena (*Kit*, top bar), unzip it, then:

```
cd hiddenhand                                # Python 3.10+, nothing to install
python backtester.py --show                  # sharp vs reader, one deal printed
cp starter_bot.py my_bot.py                 # your entry starts here
python backtester.py my_bot.py naive --deals 1000
python backtester.py --validate my_bot.py
```

11 FAQ

I have never traded. Is that a problem?

No. It is a probability game more than a finance one.

Can I use numpy or pandas?

No, standard library only. The game is small enough not to need them.

How fast must my bot be?

A millisecond or two per call. A call over 50ms counts as doing nothing.

Why did my rank change when I did not upload?

Other teams uploaded, and your bot now plays their new bots.

Where do I ask?

Your mentor first, then the WhatsApp community. Rule answers given there apply to everyone.

11.1 Glossary

PnL	Profit and loss: how much you made (positive) or lost (negative)
Long / short	Long: you bought, so you gain if the sum ends higher. Short: you sold
Lot	One unit of the stock; each lot pays the final sum
Maker / taker	The maker names the price; the taker decides whether to trade
Spread	The gap between the buy price (price + 2) and the sell price (price − 2)
Card counting	Tracking which cards are gone, so you know what is left

12 Timeline

Day	What happens	Aim to have
Mon 5 Oct	Kickoff: the game, the kit, meet your mentor. The arena opens	Run <code>python backtester.py --show</code> and explain one deal end to end
Tue 6 Oct	Mentor check-in 1	A bot that beats naive and is on the live board
Wed 7 Oct	Mentor check-in 2	A bot that beats reader ; you know where you lose PnL
Thu 8 Oct	Uploads close (exact time on the WhatsApp community). Final run, then results	Your newest upload is the one you want

All updates go out on the WhatsApp community (the QR code on the poster). Rules may be clarified during the event, and announcements there apply to everyone.